.NET Backend, Microservices & System Design Interview Handbook

Gated Funnel Format — Candidate + Interviewer Edition

> **How to use:** Start at Gate 1. Stop when the candidate hits their ceiling. Gates 2-4 are heavily scenario-based.

TABLE OF CONTENTS

GATE 1 — Fundamentals (Junior: 0-3 years)

- Q1. What is REST? What makes an API "RESTful"?
- Q2. What is middleware in ASP.NET Core?
- Q3. What are the DI lifetimes in ASP.NET Core?
- Q4. What is JWT authentication? How does it work?
- Q5. What is CORS and why is it needed?
- Q6. What is the difference between authentication and authorization?
- Q7. What is model validation? How does it work in ASP.NET Core?
- Q8. What is Swagger / OpenAPI?
- Q9. What is the Options pattern in ASP.NET Core?
- Q10. What is Entity Framework Core? How is it different from Dapper/NPoco?

GATE 2 — Working Knowledge (Mid: 3-7 years)

- Q11. Explain the ASP.NET Core middleware pipeline in detail. Why does order matter?
- Q12. Explain the MVC filter pipeline. How is it different from middleware?
- Q13. Scenario: Your API returns 401 for authenticated users on certain endpoints. What do you check?
- Q14. How does rate limiting work in .NET 7+?
- Q15. Explain caching strategies in ASP.NET Core.
- Q16. Scenario: We have M2M auth where each API call fetches a security token for validation. Latency is high. What do you do?
- Q17. What are Health Checks and how do you implement them?
- Q18. What are Background Services and when would you use them?
- Q19. What is structured logging? Why is it better than string interpolation?
- Q20. Explain OAuth2 and OpenID Connect at a high level.

GATE 3 — Deep Understanding (Senior: 7-15 years)

- Q21. Scenario: You need to write custom middleware that logs request and response bodies for debugging. What are the challenges?
- Q22. OWASP Top 10 — How does ASP.NET Core mitigate common vulnerabilities?
- Q23. Explain the Circuit Breaker pattern. When would you use Polly?

- Q24. Scenario: You're implementing a caching layer for a product catalog. How do you handle cache invalidation?
- Q25. Explain CQRS. When is it worth the complexity?
- Q26. Monolith vs Microservices — how do you decide?
- Q27. Explain the Saga pattern for distributed transactions.
- Q28. How does SignalR work? When would you use it?

GATE 4 — Architecture & Leadership (Lead/Architect: 15+ years)

- Q29. System Design: Design a notification service.
- Q30. System Design: Design a URL shortener (like bit.ly).
- Q31. System Design: Design a rate limiter.
- Q32. Scenario: Your microservices communicate via HTTP. During peak traffic, one service slowing down causes all others to slow down (cascading failure). How do you fix this?
- Q33. How do you approach API versioning in a large system?
- Q34. Scenario: You're designing a new microservices platform from scratch. What observability strategy do you implement?

APPENDIX — Backend Quick Reference

GATE 1 — Fundamentals `[Junior: 0-3 years]`

> **Style:** Direct conceptual questions. Can they explain core backend concepts?

Q1. What is REST? What makes an API "RESTful"?

Expected Answer:

REST (Representational State Transfer) is an architectural style for APIs built on HTTP.

Key principles:

- **Resources identified by URLs:** `/api/users/42`
- **HTTP verbs for actions:**

Verb	Purpose	Idempotent?	Example
GET	Read	Yes	`GET /api/users/42`
POST	Create	No	`POST /api/users` + body
PUT	Replace	Yes	`PUT /api/users/42` + body
PATCH	Partial update	Yes	`PATCH /api/users/42` + partial body
DELETE	Remove	Yes	`DELETE /api/users/42`

- **Stateless:** Each request contains all information needed. No server-side session.
- **Proper status codes:**

Code	Meaning	When
200	OK	Successful GET/PUT/PATCH
201	Created	Successful POST
204	No Content	Successful DELETE
400	Bad Request	Validation error
401	Unauthorized	Not authenticated
403	Forbidden	Authenticated but not authorized
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate or state conflict
500	Internal Server Error	Unhandled exception

Q2. What is middleware in ASP.NET Core?

Expected Answer:

Middleware is a **component in the request pipeline** that handles requests and responses. Each middleware can:

- Process the request and pass it to the next middleware.
- Short-circuit the pipeline (stop processing, return response immediately).
- Process the response on the way back.

```
app.Use(async (context, next) =>
{
    // BEFORE — request going in
    Console.WriteLine($"Request: {context.Request.Path}");

    await next();  // Call next middleware

    // AFTER — response coming back
    Console.WriteLine($"Response: {context.Response.StatusCode}");
});
```

Pipeline visualized:

```
Request  →  Middleware 1  →  Middleware 2  →  Middleware 3  →  Endpoint
Response ←  Middleware 1  ←  Middleware 2  ←  Middleware 3  ←  Endpoint
```

Q3. What are the DI lifetimes in ASP.NET Core?

Expected Answer:

Lifetime	When created	When disposed	Use for
Transient	Every time requested	When scope ends	Lightweight, stateless services
Scoped	Once per HTTP request	At end of request	Per-request state (DbContext, UserContext)
Singleton	First time requested	App shutdown	Caches, configuration, thread-safe services

```
builder.Services.AddTransient<IEmailService, SmtpEmailService>();
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddSingleton<ICacheManager, MemoryCacheManager>();
```

Key rule: A service can only depend on services with **equal or longer** lifetimes:

- Singleton → only Singleton
- Scoped → Scoped or Singleton
- Transient → anything

What to listen for: Knows all three lifetimes and the dependency rule. **Red flag:** Doesn't know what "scoped" means.

Q4. What is JWT authentication? How does it work?

Expected Answer:

JWT (JSON Web Token) is a **stateless authentication token** — the server doesn't store session data.

```
Client                          Server
  │                                │
  ├── POST /login (credentials) ──→│
  │                                ├── Validate credentials
  │                                ├── Create JWT (header.payload.signature)
  │←── 200 OK + JWT token ────────┤
  │                                │
  ├── GET /api/data ──────────────→│
  │   Authorization: Bearer eyJ...  ├── Validate JWT signature
  │                                ├── Extract claims (userId, role)
  │←── 200 OK + data ─────────────┤
```

JWT structure: `header.payload.signature`

- **Header:** Algorithm + token type
- **Payload:** Claims (userId, role, expiration)
- **Signature:** HMAC or RSA signature to prevent tampering

```
// ASP.NET Core JWT setup
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new()
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidIssuer = "your-issuer",
            IssuerSigningKey = new SymmetricSecurityKey(keyBytes)
        };
    });
```

Q5. What is CORS and why is it needed?

Expected Answer:

CORS (Cross-Origin Resource Sharing) is a browser security feature that **blocks requests from different origins** unless the server explicitly allows them.

Origin = protocol + domain + port: `https://myapp.com:443`

```
Browser at https://myapp.com
    │
    ├── Fetch GET https://api.myapp.com/data  ← DIFFERENT ORIGIN
    │
    ├── Browser sends preflight OPTIONS request
    │
    ├── Server responds with allowed origins, methods, headers
    │   Access-Control-Allow-Origin: https://myapp.com
    │
    └── If allowed → actual request proceeds
```

```
// ASP.NET Core CORS
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.WithOrigins("https://myapp.com")
              .AllowAnyMethod()
              .AllowAnyHeader();
    });
});

app.UseCors("AllowFrontend");  // Must be before UseAuthorization
```

Q6. What is the difference between authentication and authorization?

Expected Answer:

Concept	Question it answers	Examples
Authentication	"Who are you?"	JWT, cookies, OAuth, Windows auth
Authorization	"What can you do?"	Roles, policies, claims-based

```
// Authentication — verify identity
app.UseAuthentication();  // Must come before Authorization

// Authorization — check permissions
app.UseAuthorization();

// On a controller:
[Authorize]                     // Must be authenticated
[Authorize(Roles = "Admin")]    // Must have Admin role
[Authorize(Policy = "CanEditOrders")]  // Must satisfy custom policy
```

Q7. What is model validation? How does it work in ASP.NET Core?

Expected Answer:

Model validation ensures incoming request data meets expected rules before your code processes it.

Data Annotations:

```
public class CreateUserRequest
{
    [Required(ErrorMessage = "Name is required")]
    [StringLength(100, MinimumLength = 2)]
    public string Name { get; set; }

    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Range(18, 120)]
    public int Age { get; set; }
}
```

FluentValidation (preferred for complex rules):

```
public class CreateUserValidator : AbstractValidator<CreateUserRequest>
{
    public CreateUserValidator()
    {
        RuleFor(x => x.Name).NotEmpty().MaximumLength(100);
        RuleFor(x => x.Email).NotEmpty().EmailAddress();
        RuleFor(x => x.Age).InclusiveBetween(18, 120);
    }
}
```

In the controller:

```
[HttpPost]
public IActionResult CreateUser(CreateUserRequest request)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    // ... process valid request
}
```

Q8. What is Swagger / OpenAPI?

Expected Answer:

Swagger (now OpenAPI) is a **specification for describing REST APIs** — it generates interactive documentation.

```
// ASP.NET Core setup
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "My API", Version = "v1" });
});

app.UseSwagger();
app.UseSwaggerUI();
```

Benefits:

- Auto-generated API documentation from your code.
- Interactive "Try it out" testing UI.
- Client SDK generation (TypeScript, C#, Java).
- Contract-first API design.

Q9. What is the Options pattern in ASP.NET Core?

Expected Answer:

The Options pattern provides **strongly-typed access to configuration sections**.

```
// appsettings.json
{
    "EmailSettings": {
        "SmtpServer": "smtp.example.com",
        "Port": 587,
        "FromAddress": "noreply@example.com"
    }
}
```

```
// Strongly-typed class
public class EmailSettings
{
    public string SmtpServer { get; set; }
    public int Port { get; set; }
    public string FromAddress { get; set; }
}

// Registration
builder.Services.Configure<EmailSettings>(
    builder.Configuration.GetSection("EmailSettings"));

// Usage in a service
public class EmailService
{
    private readonly EmailSettings _settings;

    public EmailService(IOptions<EmailSettings> options)
    {
        _settings = options.Value;
    }
}
```

`IOptions<T>` **vs** `IOptionsSnapshot<T>` **vs** `IOptionsMonitor<T>`**:**

Interface	Lifetime	Reloads on change?
`IOptions<T>`	Singleton	No
`IOptionsSnapshot<T>`	Scoped	Yes (per request)
`IOptionsMonitor<T>`	Singleton	Yes (real-time, with change callback)

Q10. What is Entity Framework Core? How is it different from Dapper/NPoco?

Expected Answer:

Feature	EF Core (Full ORM)	Dapper/NPoco (Micro-ORM)
Query style	LINQ (C# expressions)	Raw SQL strings
Change tracking	Yes (detects modifications)	No
Migrations	Yes (schema versioning)	No
Performance	Good but overhead	Near-raw ADO.NET speed
SQL control	Less (generated SQL)	Full control
Learning curve	Higher	Lower

```
// EF Core
var users = await context.Users
    .Where(u => u.IsActive)
    .OrderBy(u => u.Name)
    .ToListAsync();

// Dapper
var users = await connection.QueryAsync<User>(
    "SELECT * FROM Users WHERE IsActive = 1 ORDER BY Name");

// NPoco
var users = await db.FetchAsync<User>(
    "WHERE IsActive = 1 ORDER BY Name");
```

When to use EF Core: CRUD-heavy apps, rapid development, schema migrations. **When to use Dapper/NPoco:** Performance-critical paths, complex SQL, legacy databases.

↑ Back to Index

GATE 2 — Working Knowledge [Mid: 3-7 years]

> **Style:** Mix of direct + scenario-based. How things work internally. Troubleshooting real problems.

Q11. Explain the ASP.NET Core middleware pipeline in detail. Why does order matter?

Expected Answer:

```
var app = builder.Build();

// ORDER MATTERS — each middleware wraps the next
app.UseExceptionHandler("/error");   // 1. Catches exceptions from everything
below
app.UseHttpsRedirection();           // 2. Redirect HTTP → HTTPS
app.UseCors("MyPolicy");             // 3. CORS headers
```

```
app.UseAuthentication();            // 4. Who are you?
app.UseAuthorization();             // 5. What can you do?
app.UseRateLimiter();               // 6. Too many requests?
app.MapControllers();               // 7. Route to controller
```

Scenario: "*We moved `UseAuthentication()` after `MapControllers()` and now all endpoints return 401. Why?*"

Answer: Auth middleware runs AFTER the endpoint is already matched. The `[Authorize]` attribute checks if the user is authenticated, but authentication never ran. Middleware must be in correct order — auth before routing.

Custom middleware example:

```
public class CorrelationIdMiddleware
{
    private readonly RequestDelegate _next;

    public CorrelationIdMiddleware(RequestDelegate next) => _next = next;

    public async Task InvokeAsync(HttpContext context)
    {
        var correlationId = context.Request.Headers["X-Correlation-Id"]
            .FirstOrDefault() ?? Guid.NewGuid().ToString();

        context.Items["CorrelationId"] = correlationId;
        context.Response.Headers["X-Correlation-Id"] = correlationId;

        await _next(context);  // Pass to next middleware
    }
}
```

Q12. Explain the MVC filter pipeline. How is it different from middleware?

Expected Answer:

Filters run **inside MVC**, not on all requests like middleware.

```
Middleware pipeline (all requests)
    └→ MVC middleware (app.MapControllers)
         └→ MVC Filter pipeline:
              1. Authorization filters  → [Authorize]
              2. Resource filters       → Caching, short-circuit
              3. Action filters         → Before/after action
              4. Exception filters      → Handle action exceptions
              5. Result filters         → Before/after result
```

Aspect	Middleware	Filters
Scope	All requests	MVC only
Access to	HttpContext	ActionContext, model binding, results
Best for	CORS, logging, auth	Validation, caching, audit

Scenario: "*Should I implement logging as middleware or a filter?*" **Answer:** Middleware — you want to log ALL requests including non-MVC (health checks, static files). Filters only run for MVC endpoints.

Q13. Scenario: Your API returns 401 for authenticated users on certain endpoints. What do you check?

Expected Answer (diagnostic checklist):

1. **Middleware order:** Is `UseAuthentication()` before `UseAuthorization()`?
2. **CORS preflight:** OPTIONS requests might be blocked → check CORS policy.
3. **Token expiry:** JWT might be expired → check `ValidateLifetime` and clock skew.
4. **Claim mismatch:** `[Authorize(Policy = "AdminOnly")]` but user lacks the claim.
5. **Scheme mismatch:** Multiple auth schemes configured but wrong `AuthenticationScheme` on the endpoint.
6. **Route-specific auth:** Some routes might have `[AllowAnonymous]` overriding class-level `[Authorize]`.

```
// Debug: Log the authentication result
app.Use(async (context, next) =>
{
    var result = await context.AuthenticateAsync();
    _logger.LogInformation("Auth result: {Succeeded}, {Failure}",
        result.Succeeded, result.Failure?.Message);
    await next();
});
```

Q14. How does rate limiting work in .NET 7+?

Expected Answer:

.NET 7 introduced built-in rate limiting middleware with four algorithms:

Algorithm	Behavior
Fixed Window	X requests per time window (e.g., 100/minute)
Sliding Window	Smooths burst at window boundaries
Token Bucket	Tokens replenish at a rate, each request consumes a token
Concurrency	Limits concurrent requests (not rate)

```
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("api", opt =>
    {
        opt.PermitLimit = 100;
        opt.Window = TimeSpan.FromMinutes(1);
        opt.QueueLimit = 10;
        opt.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
    });

    options.RejectionStatusCode = 429;  // Too Many Requests
});

app.UseRateLimiter();

// Apply to specific endpoints
[EnableRateLimiting("api")]
[HttpGet("data")]
public IActionResult GetData() { ... }
```

Scenario: "*We need rate limiting per user, not globally. How?*"

Answer: Use `PartitionedRateLimiter` with user identity as the partition key:

```
options.AddPolicy("per-user", context =>
    RateLimitPartition.GetFixedWindowLimiter(
        partitionKey: context.User.Identity?.Name ?? "anonymous",
        factory: _ => new FixedWindowRateLimiterOptions
        {
            PermitLimit = 50,
            Window = TimeSpan.FromMinutes(1)
        }));
```

Q15. Explain caching strategies in ASP.NET Core.

Expected Answer:

Strategy	Where	Scope	Best for
In-memory (`IMemoryCache`)	App process	Single instance	Hot data, session-like data
Distributed (`IDistributedCache`)	Redis/SQL	Shared across instances	Multi-instance deployments
Response caching	Client + proxy	HTTP level	Static/semi-static responses
Output caching (.NET 7+)	Server	HTTP level	Server-side response cache

```
// In-memory cache
public class ProductService
{
    private readonly IMemoryCache _cache;

    public Product GetProduct(int id)
    {
        return _cache.GetOrCreate($"product:{id}", entry =>
        {
            entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10);
            entry.SlidingExpiration = TimeSpan.FromMinutes(2);
            return _repository.GetById(id);
        });
    }
}

// Distributed cache (Redis)
public async Task<Product> GetProductAsync(int id)
{
    var cached = await _distributedCache.GetStringAsync($"product:{id}");
    if (cached != null)
        return JsonSerializer.Deserialize<Product>(cached);

    var product = await _repository.GetByIdAsync(id);
    await _distributedCache.SetStringAsync($"product:{id}",
        JsonSerializer.Serialize(product),
        new DistributedCacheEntryOptions
        {
            AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10)
        });
    return product;
}
```

Q16. Scenario: We have M2M auth where each API call fetches a security token for validation. Latency is high. What do you do?

Expected Answer:

Problem: Token validation makes a remote call on every request → 100-200ms added latency.

Solution: Cache the token/validation result.

```
public class CachedTokenValidator
{
    private readonly IMemoryCache _cache;
    private readonly ITokenService _tokenService;

    public async Task<TokenValidationResult> ValidateAsync(string token)
    {
```

```
        // Cache key based on token hash (don't cache full token for security)
        var cacheKey = $"token:{ComputeHash(token)}";

        return await _cache.GetOrCreateAsync(cacheKey, async entry =>
        {
            var result = await _tokenService.ValidateTokenAsync(token);

            // Cache until token expires (minus buffer for clock skew)
            if (result.IsValid)
            {
                var expiry = result.ExpiresAt - DateTime.UtcNow -
TimeSpan.FromMinutes(5);
                entry.AbsoluteExpirationRelativeToNow = expiry;
            }
            else
            {
                // Cache invalid results briefly to prevent repeated calls
                entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromSeconds(30);
            }

            return result;
        });
    }
}
```

What to listen for: Caches with token-expiry-aware TTL, considers security (don't log tokens), handles invalid tokens. **Red flag:** "Cache the token forever" or no consideration of expiry.

Q17. What are Health Checks and how do you implement them?

Expected Answer:

Health checks allow load balancers and orchestrators (Kubernetes) to know if your app is healthy.

```
// Registration
builder.Services.AddHealthChecks()
    .AddCheck("database", () =>
    {
        // Check database connectivity
        try { db.ExecuteScalar("SELECT 1"); return HealthCheckResult.Healthy(); }
        catch (Exception ex) { return HealthCheckResult.Unhealthy("DB down", ex);
}
    })
    .AddCheck("redis", () => /* check Redis */ )
    .AddCheck<CustomHealthCheck>("custom");

// Endpoints
app.MapHealthChecks("/health/live", new HealthCheckOptions
{
    Predicate = _ => false  // Liveness — just "is the process running?"
```

```
});

app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = _ => true  // Readiness — are dependencies available?
});
```

Liveness vs Readiness:

- **Liveness:** "Is the process alive?" → If no, restart it.
- **Readiness:** "Can it serve traffic?" → If no, remove from load balancer.

Q18. What are Background Services and when would you use them?

Expected Answer:

```
public class QueueProcessor : BackgroundService
{
    private readonly IServiceScopeFactory _scopeFactory;

    public QueueProcessor(IServiceScopeFactory scopeFactory)
        => _scopeFactory = scopeFactory;

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            using var scope = _scopeFactory.CreateScope();
            var processor =
scope.ServiceProvider.GetRequiredService<IMessageProcessor>();

            await processor.ProcessNextAsync(stoppingToken);
            await Task.Delay(TimeSpan.FromSeconds(5), stoppingToken);
        }
    }
}

// Register
builder.Services.AddHostedService<QueueProcessor>();
```

Use cases: Message queue consumers, scheduled tasks, cache warming, health monitoring.

Key gotcha: Background services are **singletons** — must create scopes to use scoped services (like DbContext).

Q19. What is structured logging? Why is it better than string interpolation?

Expected Answer:

```
// BAD — string interpolation (unstructured)
_logger.LogInformation($"Order {orderId} processed for user {userId} in
{elapsed}ms");
// Produces: "Order 42 processed for user alice in 150ms"
// Can't search by orderId in log aggregator!

// GOOD — structured logging (message template)
_logger.LogInformation("Order {OrderId} processed for user {UserId} in
{ElapsedMs}ms",
    orderId, userId, elapsed);
// Produces structured data: { OrderId: 42, UserId: "alice", ElapsedMs: 150 }
// Searchable, filterable, dashboardable in Datadog/ELK/Splunk!
```

Benefits:

- **Queryable:** Find all logs where `OrderId = 42`.
- **Aggregatable:** Average `ElapsedMs` across all orders.
- **Alertable:** Alert when `ElapsedMs > 5000`.

Q20. Explain OAuth2 and OpenID Connect at a high level.

Expected Answer:

OAuth2: Authorization framework — "Allow App X to access my data on Service Y." **OpenID Connect (OIDC):** Identity layer on top of OAuth2 — "Also tell App X who I am."

```
User → App → Identity Provider (Auth0, Azure AD, Okta)
                    │
                    ├── OAuth2: "Here's an access_token to call APIs"
                    └── OIDC:   "Here's an id_token with user info (name, email,
roles)"
```

OAuth2 Flows:

Flow	Used by	How
Authorization Code + PKCE	Web apps, SPAs, mobile	Redirect to IdP, get code, exchange for token
Client Credentials	Machine-to-machine (M2M)	App sends client_id + secret, gets token directly
Device Code	Smart TVs, CLI tools	Display code on screen, user authorizes on phone

↑ Back to Index

GATE 3 — Deep Understanding [Senior: 7-15 years]

> **Style:** Mostly scenario-based. Expect diagnosis, trade-off analysis, and implementation details.

Q21. Scenario: You need to write custom middleware that logs request and response bodies for debugging. What are the challenges?

Expected Answer:

Challenges:

1. **Request body is forward-only** — once read, it's gone. Must enable buffering.
2. **Response body is write-only** — can't read it after the response is sent. Must swap the stream.
3. **Performance** — Logging bodies adds latency and memory pressure.
4. **Security** — Don't log sensitive data (passwords, tokens, PII).

```
public class RequestResponseLoggingMiddleware
{
    private readonly RequestDelegate _next;

    public async Task InvokeAsync(HttpContext context)
    {
        // Enable request body buffering
        context.Request.EnableBuffering();

        // Read request body
        var requestBody = await new
StreamReader(context.Request.Body).ReadToEndAsync();
        context.Request.Body.Position = 0;  // Reset for downstream

        // Capture response body by swapping the stream
        var originalResponseBody = context.Response.Body;
        using var memoryStream = new MemoryStream();
        context.Response.Body = memoryStream;

        await _next(context);

        // Read response body
        memoryStream.Position = 0;
        var responseBody = await new StreamReader(memoryStream).ReadToEndAsync();
        memoryStream.Position = 0;

        // Copy back to original stream
        await memoryStream.CopyToAsync(originalResponseBody);
        context.Response.Body = originalResponseBody;

        _logger.LogInformation("Request: {Path} Body: {RequestBody} Response:
{ResponseBody}",
```

```
                context.Request.Path, Sanitize(requestBody), Sanitize(responseBody));
        }

        private string Sanitize(string body)
        {
            // Remove passwords, tokens, PII before logging
        }
    }
```

Q22. OWASP Top 10 — How does ASP.NET Core mitigate common vulnerabilities?

Expected Answer:

Vulnerability	Attack	.NET Mitigation
SQL Injection	`'; DROP TABLE Users;--`	Parameterized queries (`@0` parameters in NPoco/Dapper), EF Core LINQ
XSS	`<script>alert('hack')</script>`	Razor auto-encodes output, `@Html.Raw()` requires explicit opt-in
CSRF	Forged form submission from another site	`[ValidateAntiForgeryToken]`, automatic in Razor Pages
Broken Auth	Weak tokens, no expiry	ASP.NET Identity, JWT validation, HTTPS enforcement
Sensitive Data Exposure	Leaking connection strings	`UserSecrets` (dev), Azure Key Vault (prod), HTTPS
Security Misconfiguration	Detailed error pages in prod	`app.UseExceptionHandler()` (not `UseDeveloperExceptionPage` in prod)
Injection (general)	OS command injection	Avoid `Process.Start` with user input, use parameterized queries

```
// SQL Injection — SAFE
var user = db.SingleOrDefault<User>("WHERE Id = @0", userId);

// SQL Injection — DANGEROUS
var user = db.SingleOrDefault<User>($"WHERE Id = {userId}");  // Never!
```

Q23. Explain the Circuit Breaker pattern. When would you use Polly?

Expected Answer:

Circuit Breaker prevents cascading failures by **stopping calls to a failing service**.

```
States:
┌─────────┐    failures > threshold    ┌──────┐
│ CLOSED  │ ──────────────────────────→ │ OPEN │
│(normal) │                             │(fail)│
└─────────┘                             └──────┘
     ↑                                     │
     │         timeout expires             │
     │                                     ▼
     │                                 ┌────────┐
     └──────── success ────────────────│ HALF   │
                                       │ OPEN   │
                                       └────────┘
```

```
// Polly circuit breaker
builder.Services.AddHttpClient("PaymentService")
    .AddPolicyHandler(Policy
        .HandleResult<HttpResponseMessage>(r => !r.IsSuccessStatusCode)
        .CircuitBreakerAsync(
            handledEventsAllowedBeforeBreaking: 5,   // 5 failures
            durationOfBreak: TimeSpan.FromSeconds(30), // Then open for 30s
            onBreak: (result, duration) =>
                _logger.LogWarning("Circuit opened for {Duration}", duration),
            onReset: () =>
                _logger.LogInformation("Circuit closed — service recovered")
        ));
```

Polly resilience policies:

- **Retry** — Retry failed calls with exponential backoff.
- **Circuit Breaker** — Stop calling a failing service.
- **Bulkhead** — Limit concurrent calls to a service.
- **Timeout** — Fail if call takes too long.
- **Fallback** — Return a default value on failure.

Q24. Scenario: You're implementing a caching layer for a product catalog. How do you handle cache invalidation?

Expected Answer:

"There are only two hard things in computer science: cache invalidation and naming things."

Strategies:

Strategy	How	Best for
TTL (Time-To-Live)	Cache expires after X minutes	Data that's OK to be slightly stale

Strategy	How	Best for
Cache-aside with invalidation	Delete cache entry when data changes	Write-heavy with consistency needs
Write-through	Update cache AND database together	Strong consistency required
Event-driven	Publish event on change, subscriber invalidates cache	Microservices, distributed systems

```
// Cache-aside with invalidation
public class ProductService
{
    public async Task<Product> GetProductAsync(int id)
    {
        var cacheKey = $"product:{id}";
        var cached = await _cache.GetAsync<Product>(cacheKey);
        if (cached != null) return cached;

        var product = await _repository.GetByIdAsync(id);
        await _cache.SetAsync(cacheKey, product, TimeSpan.FromMinutes(10));
        return product;
    }

    public async Task UpdateProductAsync(Product product)
    {
        await _repository.UpdateAsync(product);
        await _cache.RemoveAsync($"product:{product.Id}");  // Invalidate
    }
}
```

What to listen for: Discusses trade-offs between consistency and performance. Mentions cache stampede (many requests hit DB simultaneously when cache expires). **Cache stampede fix:** Use a lock or `GetOrCreate` pattern to ensure only one request refreshes the cache.

Q25. Explain CQRS. When is it worth the complexity?

Expected Answer:

CQRS (Command Query Responsibility Segregation): Separate the read model from the write model.

```
Traditional:
    Service → Repository → Single Database Model

CQRS:
    Command (Write) → Command Handler → Write Model → Write DB
    Query (Read)    → Query Handler   → Read Model  → Read DB (optimized view)
```

```
// With MediatR
public record CreateOrderCommand(string CustomerId, List<OrderItem> Items) :
IRequest<int>;
public record GetOrderQuery(int OrderId) : IRequest<OrderDto>;

public class CreateOrderHandler : IRequestHandler<CreateOrderCommand, int>
{
    public async Task<int> Handle(CreateOrderCommand command, CancellationToken
ct)
    {
        var order = Order.Create(command.CustomerId, command.Items);
        await _writeRepository.SaveAsync(order);
        await _eventBus.Publish(new OrderCreatedEvent(order.Id));
        return order.Id;
    }
}

public class GetOrderHandler : IRequestHandler<GetOrderQuery, OrderDto>
{
    public async Task<OrderDto> Handle(GetOrderQuery query, CancellationToken ct)
    {
        // Reads from denormalized, query-optimized view
        return await _readRepository.GetOrderViewAsync(query.OrderId);
    }
}
```

When CQRS is worth it:

- Read and write patterns are very different (e.g., complex writes, simple reads).
- Read and write scale differently (10x more reads than writes).
- You need different models for reads (denormalized views) and writes (normalized entities).

When CQRS is overkill:

- Simple CRUD apps.
- Read and write patterns are similar.
- Small team — added complexity isn't worth it.

Q26. Monolith vs Microservices — how do you decide?

Expected Answer:

Factor	Monolith	Microservices
Team size	< 10 developers	10+ developers across teams
Deployment	Deploy everything together	Deploy services independently
Complexity	Simple to develop, test, debug	Distributed systems complexity (network, consistency)

Factor	Monolith	Microservices
Scaling	Scale the whole app	Scale individual services
Data	Shared database	Database per service
Communication	Method calls (in-process)	HTTP/gRPC/messaging (network)

Start with a monolith when:

- New product, uncertain requirements.
- Small team.
- Need to move fast.

Consider microservices when:

- Multiple teams need independent deployment.
- Different parts need different scaling.
- Clear bounded contexts.

Scenario: "*The CTO says we need microservices because Netflix uses them.*"

Answer: "Netflix has thousands of engineers. We have 8. A well-structured monolith (modular monolith) with clear boundaries is often better for our team size. We can extract services later when we have a concrete scaling or team-organization need."

Q27. Explain the Saga pattern for distributed transactions.

Expected Answer:

In microservices, you can't use database transactions across services. Saga manages distributed workflows through a sequence of local transactions with compensating actions.

Two approaches:

Choreography (event-driven):

```
Order Service: Create Order → Publish OrderCreated
    ↓
Payment Service: Listens → Charge payment → Publish PaymentCompleted
    ↓
Inventory Service: Listens → Reserve stock → Publish StockReserved
    ↓
Order Service: Listens → Confirm order

If Payment fails:
Payment Service: Publish PaymentFailed
    ↓
Order Service: Listens → Cancel order (compensating action)
```

Orchestration (central coordinator):

```
Saga Orchestrator:
    1. Tell Order Service: Create order
    2. Tell Payment Service: Charge payment
    3. Tell Inventory Service: Reserve stock
    4. If step 2 fails → Tell Order Service: Cancel order
    5. If all succeed → Tell Order Service: Confirm order
```

Approach	Pros	Cons
Choreography	Decoupled, no single point of failure	Hard to understand flow, debugging is painful
Orchestration	Clear flow, easy to understand	Central coordinator is a bottleneck/SPOF

Q28. How does SignalR work? When would you use it?

Expected Answer:

SignalR enables **real-time, bidirectional communication** between server and client.

Transport negotiation: WebSockets → Server-Sent Events → Long Polling (falls back automatically).

```dotnet
// Server — Hub
public class NotificationHub : Hub
{
    public async Task SendToUser(string userId, string message)
    {
        await Clients.User(userId).SendAsync("ReceiveNotification", message);
    }

    public async Task JoinGroup(string groupName)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId, groupName);
    }

    public async Task SendToGroup(string groupName, string message)
    {
        await Clients.Group(groupName).SendAsync("ReceiveMessage", message);
    }
}

// Registration
builder.Services.AddSignalR();
app.MapHub<NotificationHub>("/hubs/notifications");
```

Use cases: Live notifications, chat, dashboards, collaborative editing, real-time game state. **Scaling challenge:** Sticky sessions or a backplane (Redis, Azure SignalR Service) for multi-instance.

↑ Back to Index

GATE 4 — Architecture & Leadership [Lead/Architect: 15+ years]

> **Style:** System design + trade-off analysis. Open-ended, expect the candidate to drive the conversation.

Q29. System Design: Design a notification service.

Expected Answer (framework):

Step 1 — Requirements gathering (candidate should ask):

- What types? Push, email, SMS, in-app?
- Volume? 100/day vs 1M/day changes the architecture.
- Latency requirements? Immediate or within minutes?
- Retry on failure? How many times?
- User preferences (opt-out)?

Step 2 — High-level design:

```
API Gateway
    │
    ▼
Notification API (accepts notification requests)
    │
    ▼
Message Queue (RabbitMQ / Azure Service Bus / Kafka)
    │
    ├─→ Email Worker → SendGrid / SMTP
    ├─→ SMS Worker → Twilio
    ├─→ Push Worker → FCM / APNS
    └─→ In-App Worker → SignalR Hub
    │
    ▼
Notification Database (delivery status, history)
    │
    ▼
User Preference Service (opt-out, channel preferences)
```

Step 3 — Key design decisions:

- **Queue per channel** vs **single queue with routing** — Queue per channel for independent scaling.
- **At-least-once delivery** — Queue guarantees with idempotency keys to prevent duplicate notifications.
- **Priority queues** — Password reset emails should be immediate, marketing can wait.
- **Rate limiting per user** — Don't spam users with 100 notifications in a minute.

- **Template engine** — Centralized templates for consistent branding.

Step 4 — Scaling:

- Workers are stateless → horizontal scaling.
- Database: read replicas for notification history.
- Queue partitioning by user ID for ordering guarantees.

Q30. System Design: Design a URL shortener (like bit.ly).

Expected Answer:

Requirements:

- Shorten a URL → return short URL.
- Redirect short URL → original URL.
- Analytics (click count, geographic data).
- Scale: 100M URLs, 1B redirects/month.

Core algorithm:

```
Long URL → Hash/Encode → Short Code → Store mapping

Option 1: Base62 encoding of auto-increment ID
    ID 12345 → base62 → "3D7" → short.ly/3D7

Option 2: Random 7-character code
    Generate random base62 string, check for collisions
```

Architecture:

```
Create:
    POST /api/shorten { url: "https://very-long-url.com/..." }
        → Generate short code
        → Store in DB: { code: "3D7", url: "https://...", created: now }
        → Return "https://short.ly/3D7"

Redirect:
    GET /3D7
        → Look up in Cache (Redis)
        → If miss, look up in DB
        → 302 Redirect to original URL
        → Async: log click analytics
```

Database: Write-heavy for creates, read-heavy for redirects.

- **Cache layer (Redis):** Cache popular URLs. 80/20 rule — 20% of URLs get 80% of traffic.
- **Database:** Key-value store (DynamoDB) or SQL with indexing on short code.

Scaling consideration: At 1B redirects/month (~400 QPS), cache hit rate is critical. Redis can handle 100K+ QPS.

Q31. System Design: Design a rate limiter.

Expected Answer:

Algorithms:

Algorithm	How	Trade-off
Fixed Window	Count requests per time window	Simple but burst at boundaries
Sliding Window Log	Track timestamp of each request	Accurate but memory intensive
Sliding Window Counter	Weighted count across windows	Good balance
Token Bucket	Tokens added at rate R, each request takes a token	Allows controlled bursts
Leaky Bucket	Requests processed at constant rate, excess queued	Smooth output, no bursts

Token Bucket implementation (distributed with Redis):

```
Key: rate_limit:{user_id}
Value: { tokens: 10, last_refill: timestamp }

On request:
    1. Calculate tokens to add since last_refill
    2. Add tokens (cap at max bucket size)
    3. If tokens >= 1: decrement, allow request
    4. Else: reject with 429 + Retry-After header

Redis Lua script for atomicity:
    EVALSHA <script> 1 rate_limit:user123
```

Architecture for distributed rate limiting:

```
API Gateway (Nginx / YARP)
    │
    ├─→ Rate Limiter (Redis-backed)
    │       └── Lua scripts for atomic token bucket
    │
    ├─→ If allowed → Backend Service
    └─→ If rejected → 429 Too Many Requests
```

Q32. Scenario: Your microservices communicate via HTTP. During peak traffic, one service slowing down causes all others to slow down (cascading failure). How do you fix this?

Expected Answer (layered resilience):

1. **Timeouts** — Never wait forever. Set HTTP client timeouts aggressively.

```
builder.Services.AddHttpClient("OrderService", c => c.Timeout =
TimeSpan.FromSeconds(5));
```

2. **Circuit Breaker** — Stop calling a failing service.

```
.AddPolicyHandler(Policy.Handle<HttpRequestException>()
    .CircuitBreakerAsync(5, TimeSpan.FromSeconds(30)));
```

3. **Bulkhead** — Isolate resource pools per downstream service.

```
.AddPolicyHandler(Policy.BulkheadAsync<HttpResponseMessage>(10, 20));
```

4. **Retry with backoff** — Retry transient failures with jitter.

```
.AddPolicyHandler(Policy.Handle<HttpRequestException>()
    .WaitAndRetryAsync(3, attempt =>
        TimeSpan.FromMilliseconds(Math.Pow(2, attempt) * 100)
        + TimeSpan.FromMilliseconds(Random.Shared.Next(0, 100))));
```

5. **Fallback** — Degrade gracefully.

```
.AddPolicyHandler(Policy.Handle<BrokenCircuitException>()
    .FallbackAsync(new HttpResponseMessage(HttpStatusCode.OK)
    {
        Content = new StringContent("{\"cached\": true}")
    }));
```

6. **Async communication** — For non-critical paths, switch from HTTP to message queue (fire-and-forget).

What to listen for: Layered approach, not just one solution. Understands that each layer handles different failure modes.

Q33. How do you approach API versioning in a large system?

Expected Answer:

Strategies:

Method	Example	Pros	Cons
URL path	`/api/v1/users`	Clear, easy to route	URL changes
Query string	`/api/users?version=2`	URL stays same	Easy to miss
Header	`Api-Version: 2`	Clean URLs	Less visible
Content negotiation	`Accept: application/vnd.api.v2+json`	RESTful	Complex

In ASP.NET Core:

```
builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
    options.ApiVersionReader = ApiVersionReader.Combine(
        new UrlSegmentApiVersionReader(),
        new HeaderApiVersionReader("X-Api-Version"));
});

[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/users")]
public class UsersV1Controller : ControllerBase { }

[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/users")]
public class UsersV2Controller : ControllerBase { }
```

Deprecation strategy:

1. Release v2 alongside v1.
2. Mark v1 as deprecated (`[ApiVersion("1.0", Deprecated = true)]`).
3. Monitor v1 usage. Communicate timeline to consumers.
4. After migration period, remove v1.

Q34. Scenario: You're designing a new microservices platform from scratch. What observability strategy do you implement?

Expected Answer:

Three pillars:

Pillar	Purpose	Tools
Logs	Discrete events	Serilog → Datadog/ELK
Metrics	Numerical measurements	Prometheus/Grafana, App Insights
Traces	Request flow across services	OpenTelemetry → Jaeger/Zipkin

Implementation:

```
// OpenTelemetry setup
builder.Services.AddOpenTelemetry()
    .WithTracing(b => b

.SetResourceBuilder(ResourceBuilder.CreateDefault().AddService("OrderService"))
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddSqlClientInstrumentation()
        .AddOtlpExporter())
    .WithMetrics(b => b
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddPrometheusExporter());
```

Key metrics to track:

- **RED method:** Rate, Errors, Duration per service.
- **USE method:** Utilization, Saturation, Errors per resource (CPU, memory, disk).
- **Business metrics:** Orders processed, payment failures, user signups.

Correlation:

- Every request gets a `CorrelationId` in middleware.
- Propagated via HTTP headers across services.
- All logs, metrics, and traces include the correlation ID.
- One click in Datadog/Jaeger shows the entire request journey.

↑ Back to Index

APPENDIX — Backend Quick Reference

Middleware Order (Recommended)

```
app.UseExceptionHandler();
app.UseHsts();
app.UseHttpsRedirection();
app.UseCors();
app.UseAuthentication();
```

```
app.UseAuthorization();
app.UseRateLimiter();
app.UseResponseCaching();
app.MapControllers();
```

HTTP Status Code Cheat Sheet

Code	Name	Use When
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST (include Location header)
204	No Content	Successful DELETE
400	Bad Request	Validation error
401	Unauthorized	Not authenticated
403	Forbidden	Authenticated but not authorized
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate or state conflict
422	Unprocessable Entity	Semantic validation error
429	Too Many Requests	Rate limited
500	Internal Server Error	Unhandled exception
502	Bad Gateway	Upstream service error
503	Service Unavailable	Overloaded or maintenance

> **Gate Assessment Summary:**
>
> | Gate | Topic Focus | Passed → Level |
> |---|---|---|
> | Gate 1 | REST, middleware, DI, JWT, CORS, validation | Junior |
> | Gate 2 | Pipeline internals, caching, rate limiting, OAuth2, EF Core | Mid |
> | Gate 3 | Custom middleware, OWASP, CQRS, Saga, resilience patterns | Senior |
> | Gate 4 | System design, API versioning, observability, distributed architecture | Lead/Architect |

↑ Back to Index